#6 Compares a point to a node, returning `0` if the node's point matches the first argument, a value lower than `0` if the point is on the "left" of the node, greater than `0` otherwise.
#7 The `sign` function returns the sign of a numeric value: `-1` for negative values, `+1` for positive ones, or `0`.
#8 Computes the distance between a point and its projection on the split line passing through a node.
#9 This distance is nothing else than the absolute value of the difference between the `j`-th coordinates of the two points, where `j = node.level mod k`.

Listing 9.2 shows a few helper functions that will help us keeping our code clean. We encapsulate in these functions the logic of cycling through split coordinates while traversing the tree, instead of duplicating it across all the methods: this way the other methods will be more readable, and if we ever have to change the way this comparison is done (f.i. because we find a bug or we want to implement a fancier algorithm) we just need to touch one single place in the code base.